

學號：11220105 姓名：楊耀甯

一、開發環境

本次的開發環境，是在Windows環境上建立一個WSL子系統(Debian)，並於當中安裝g++編譯器(g++ 14.2.0)編譯C++程式。

二、實作方法與流程

本次作業的實作分為 4 個部分：

1、將N筆數目字直接進行BubbleSort，並顯示CPU執行之時間。

時間複雜度： $O(N^2)$ 。

步驟：

- (1) 從txt檔讀取資料到陣列中。
- (2) 直接針對整筆資料進行BubbleSort($O(N^2)$)。
- (3) 把排序好的陣列資料存進新檔案中。

2、將N筆數目字切成K份，先在一个process內對K份資料進行BubbleSort之後，再用同一個process作MergeSort，並顯示CPU執行之時間。

時間複雜度： $O(N^2 / K + N * \log K)$ 。

步驟：

- (1) 從txt檔讀取資料到陣列中。
- (2) 根據使用者輸入的slices數來決定要把陣列平均切成幾塊。
- (3) 利用迴圈把每個區段一一進行BubbleSort，使每個區段都各自排序好(

$$O(N^2 / K) = O((N^2 / K^2) * K)。$$

- (4) 利用TreeMerge，把各個區段兩兩合併成一個排序好的區段，藉由一輪輪的合併(採用遞迴)，最終把整筆資料合併排序好(如圖 1， $O(N * \log K)$)。

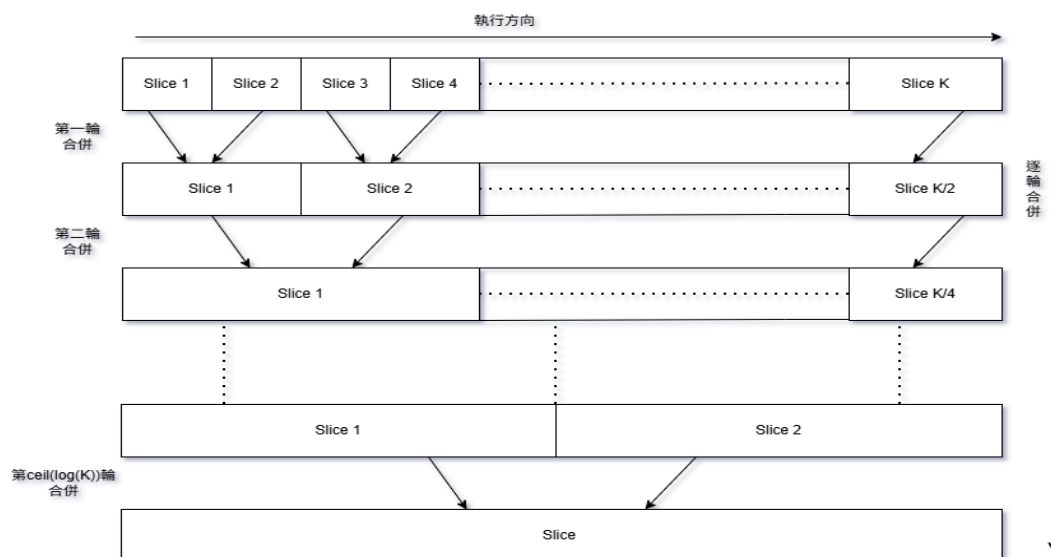


圖 1：TreeMerge示意圖，時間複雜度 $O(N * \log(K))$ 。

- (5) 把排序好的陣列資料存進新檔案中。

3、將N筆數目字切成K份，並由K個processes各別進行 BubbleSort 之後，再用K-1 個 process(es) MergeSort，並顯示CPU執行之時間。

時間複雜度： $O(N^2 / K^2 + N * \log K)$ 。

建立shared memory的目的、方法：

建立shared memory目的在於給許多個processes共用一個儲存空間，首先，我們會先把主process建立好的陣列複製到shared memory，接著，再給processes進行BubbleSort與MergeSort。在C++中，我們採用<sys/mman.h>所提供的mmap()來建立shared memory，並且宣告一pointer使其指向shared memory。

建立、結束process的方法：

在C++，我們利用<unistd.h>提供的fork()，來建立child process，藉由建立許多child processes，來完成個區段的排序，最後當所有process都工作完時，利用<sys/wait.h>的waitpid()來回收child processes並等待他們完全結束，這樣就不會有zombie process的問題。

步驟：

- (1) 從txt檔讀取資料到主process的陣列中。
- (2) 根據使用者輸入的slices數來決定要把陣列平均切成幾塊。
- (3) 建立shared memory，給多個process使用，並把讀取到主process的資料存進shared memory。
- (4) 利用迴圈把shared memory的每個區段各分配給一個process進行BubbleSort，使每個區段都各自排序好($O(N^2 / K^2)$)。
- (5) 利用TreeMerge，把每個shared memory區段各藉由一個process兩兩合併成一個排序好的區段，透過一輪輪合併(採用遞迴)，最終把整筆資料合併排序好($O(N * \log K)$)。
- (6) 把shared memory中排序好的資料存回主process的陣列，再把其存進新檔案中。

4、將N筆數目字切成K份，並由K個threads各別進行 BubbleSort 之後，再用K-1 個 thread(s) 作MergeSort，並顯示CPU執行之時間。

時間複雜度： $O(N^2 / K^2 + N * \log K)$ 。

建立、結束thread的方法：

在C++，我們使用<thread>來建立 threads。BubbleSort 與 MergeSort 都是透過迴圈建立多個 threads，並讓每個 thread 負責其對應的資料區段。當 BubbleSort 的 threads 建立完後，會透過join()等待並回收所有 threads，避免進入 MergeSort 時發生 race condition。同樣地，MergeSort 在所有合併任務完成後，也會使用join()回收 threads，以確保所有工作都正確結束。另外，threads不同於processes，不需要shared memory，因為threads有共同的数据 section，並且都是同時屬於一個process管轄範圍)。

步驟：

- (1) 從txt檔讀取資料到主process的陣列中。
- (2) 根據使用者輸入的slices數來決定要把陣列平均切成幾塊。
- (3) 利用迴圈把陣列的每個區段各分配給一個thread進行BubbleSort，使每個區段都各自排序好($O(N^2 / K^2)$)。
- (4) 利用TreeMerge，把每個陣列區段各藉由一個thread兩兩合併成一個排序好的區段，透過一輪輪合併(採用遞迴)，最終把整筆資料合併排序好($O(N * \log K)$)。
- (5) 把排序好的陣列資料存進新檔案中。

三、探討結果與原因

$K =$ {20, 100, 5000, 10000}	$N = 10K$	$N = 100K$	$N = 500K$	$N = 1M$
方法一($K = 1$)	347.922	36587.134	914675.236	4014792.393
方法二	19.832, 7.899, 7.654, 6.552	1633.271, 330.033, 45.828, 39.961	45951.401, 8091.465, 313.256, 230.346	223198.083, 35218.212, 897.379, 612.604
方法三	17.301, 48.926, 955.074, 1928.916	156.908, 61.311, 1043.16, 2090.53	4411.022, 748.173, 1139.71, 2299.72	19027.46, 3341.26, 1344.20, 2648.21
方法四	13.526, 30.199, 401.189, 831.182	285.796, 70.819, 428.107, 836.780	12980.286, 1318.643, 463.459, 875.401	30266.019, 7578.150, 532.930, 965.595

表1：相同資料筆數 N ，不同 K 值的執行時間。(單位： ms)

$N =$ {10K, 100K, 500K, 1M}	$K = 20$	$K = 100$	$K = 5000$	$K = 10000$
方法一 ($K = 1$)	347.922, 36587.134, 914675.236, 4014792.393			
方法二	19.832, 1633.271, 45951.401, 223198.083	5.238, 330.033, 8091.465, 35218.212	7.035, 45.828, 313.256, 897.379	5.265, 39.961, 230.346, 612.604
方法三	19.048, 156.908, 4411.022, 19027.46	58.014, 61.311, 748.173, 3341.26	1042.34, 1043.16, 1139.71, 1344.20	2051.68, 2090.53, 2299.72, 2648.21
方法四	17.287, 285.796, 12980.286, 30266.019	13.327, 70.819, 1318.643, 7578.150	414.595, 428.107, 463.459, 532.930,	822.879, 836.780, 875.401, 965.595

表2：相同 K 值，不同資料筆數 N 的執行時間。(單位： ms)

表1的部分我們逐一分析，首先，**方法一的部分**，很明顯可以從中看出執行時間是跟著 $O(N^2)$ 的趨勢成長，像是這四筆資料的執行時間比就大約等於 $1^2:10^2:50^2:100^2$ ；再者，**方法二的部分**，我們可以看到四個欄位的執行時間都明顯有個趨勢，就是切的區段數K越多，其執行時間就越短，很符合 $O(N^2/K)$ 的趨勢，至於在 $O(N * \log(K))$ 的部分，由於 $\log(K)$ 的成長趨勢比K還小，所以對於時間變化較不會影響；至於**方法三與方法四**情況就比較特殊，由於processes與threads在建立以及結束、回收皆需要花費時間，因此特別是此二方法在排10K筆資料時，執行時間會隨著K值的變大而跟著變多，就是因為時間佔比很大一部份都放在process與thread的建立與結束上，故不會符合理想情況下的

$O(N^2/K^2 + N * \log K)$ 時間複雜度，另外在方法三排序100K、500K、1M筆資料時，執行時間從K = 20到K = 100有變短，是因為process與thread的數量不足以大量影響執行時間，然而K = 5000與K = 10000時，由於此時processes與threads的數量大幅增加，所以執行時間也就跟著增加許多，而根據圖表，我們也觀測到當K值與N值皆夠大時(方法四的500K與1M)，其執行時間就會如同方法二那四個欄位一樣，都是隨著K值增加而減少。

至於表 2 的整體趨勢，我們可以看到從左上到右下，欄位中的四組時間會越來越接近。也就是說，當K（切片數）開得越大、並且使用 multi-process 或 multi-thread 時，不管資料量 N 是多少，執行時間最後都變得差不多。因此我們可以推論：切太多片之後，排序本身已經不是主要花時間的地方。這時候真正拖慢速度的是process與thread 的建立和結束、CPU 做 context switch、要等 waitpid() 或 join()、shared memory建立或合併的處理等等。而這些開銷跟資料量 N 完全無關，只跟切了多少個 K 有關。當每片資料少到 BubbleSort 幾乎不需要時間時，N 的大小就不再影響結果了，不同 N 的時間自然會變得很接近。這也代表，切片切太多並不會加速，反而因為管理成本太高，導致執行時間都集中在同一個區間，甚至比少切片時還慢。

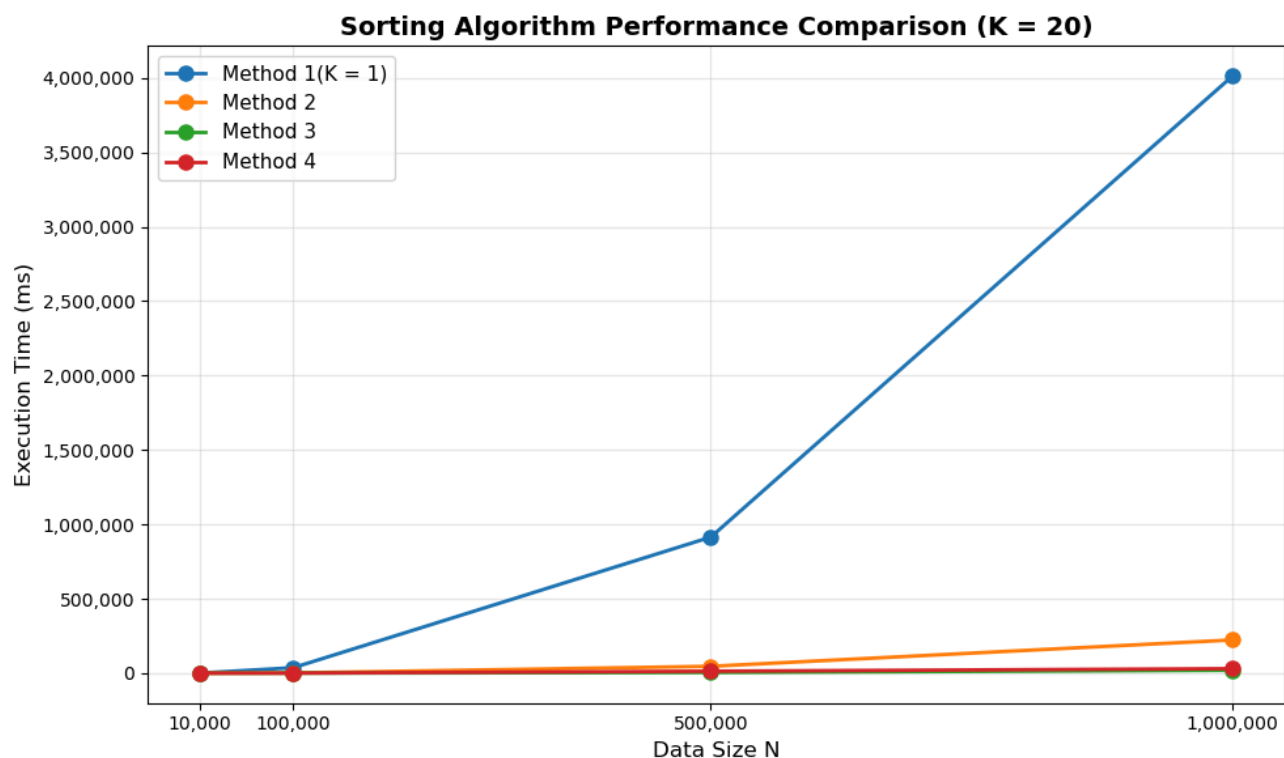


表3：四種方法的執行效能比較圖(以K = 20為例)。

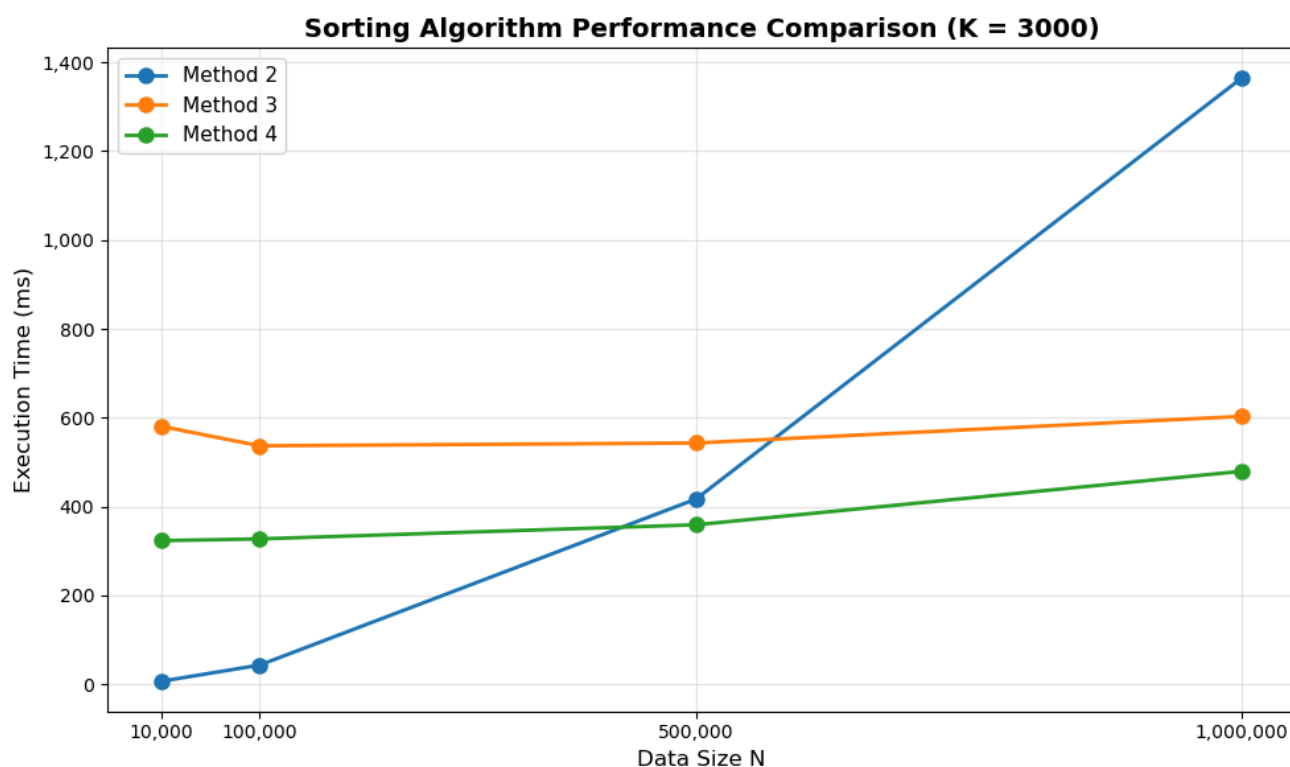


表4：方法二、方法三、方法四的執行效能比較圖(以K = 3000為例)。

最後是表3與表4的部分，我們可以看到表3方法一的曲線真的就如同 $O(N^2)$ 一般，執行時間會隨著資料筆數的增加而跟著平方式的增長，至於方法二、三、四，我們可以觀察到，在資料筆數較小時，執行時間是差不多的，但是隨著資料筆數的增加，到500000以上時，我們就可以逐漸發現方法二(single process)的執行時間逐漸比方法三(multi-process)與方法四(multi-thread)來的多，這是因為方法三與方法四將資料的每個區段分派給不同的processes與threads排序，因此執行時間上會比方法二來的短，至於先前提到過的process與thread的建立與回收的時間問題，在表3無法明顯被看出，因為以表3來說，K值只有20，也就是在進行BubbleSort與MergeSort時，分別只有生成20個和19個processes與threads，因此比較不會有上述問題，不過類似問題就可以明顯的在表4中看出，表4中明顯顯示無論資料筆數為何，由於K = 3000的關係，因此生成的processes與threads數量就已經達到3000個的量級，此時執行時間很大一部份是取決於processes與threads的建立，而不是在於演算法，同時我們也可以看出，方法三的執行時間皆長於方法四，那是因為建立一個process的時間比建立一個thread來的長，所以才會有更長的執行時間。